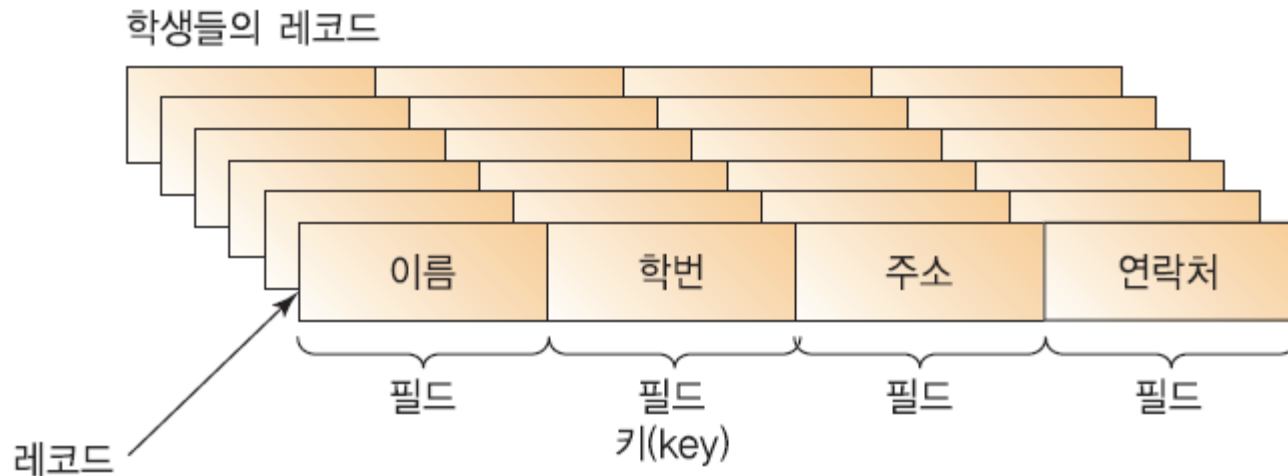


정 렬

정렬의 대상

- 일반적으로 정렬시켜야 될 대상은 레코드(record)
- 레코드는 필드(field)라는 보다 작은 단위로 구성
- 키필드로 레코드와 레코드를 구별한다.



정렬 알고리즘 개요

- 모든 경우에 최적인 정렬 알고리즘은 없음
- 각 응용 분야에 적합한 정렬 방법 사용해야함
 - ⊙ 레코드 수의 많고 적음
 - ⊙ 레코드 크기의 크고 작음
 - ⊙ Key의 특성(문자, 정수, 실수 등)
 - ⊙ 메모리 내부/외부 정렬
- 정렬 알고리즘의 평가 기준
 - ⊙ 비교 횟수의 많고 적음
 - ⊙ 이동 횟수의 많고 적음

정렬 알고리즘 개요

- 단순하지만 비효율적인 방법
 - ⊙ 삽입정렬, 선택정렬, 버블정렬 등
- 복잡하지만 효율적인 방법
 - ⊙ 퀵정렬, 힙정렬, 합병정렬, 기수정렬 등

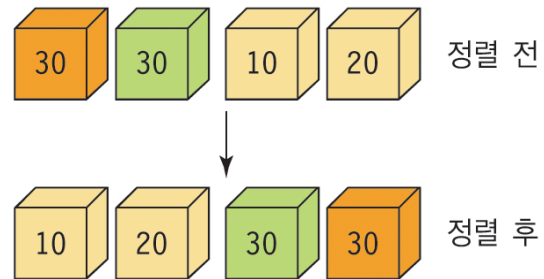
정렬 알고리즘 개요

- 정렬 알고리즘의 안정성(stability)

- ⊙ 동일한 키 값을 갖는 레코드들의 상대적인 위치가 정렬 후에도 바뀌지 않음

- ⊙ 안정하지 않은 정렬의 예

=====>



선택정렬(selection sort)

- 정렬된 왼쪽 리스트와 정렬 안된 오른쪽 리스트 가정
 - ⊙ 초기에는 왼쪽 리스트는 비어 있고, 정렬할 숫자들은 모두 오른쪽 리스트에 존재
- 오른쪽 리스트에서 최소값 선택하여 오른쪽 리스트의 첫번째 수와 교환
 - ⊙ 왼쪽 리스트 크기 1 증가
 - ⊙ 오른쪽 리스트 크기 1 감소
 - ⊙ 오른쪽 리스트가 소진되면 정렬 완료



선택정렬

```
1. Algorithm Selection_sort(random_data, n)
2. Input: 양의 정수를 갖는 데이터 배열(random_data), 데이터 수(n)
3. Output: 오름차순으로 정렬된 배열(random_data)
4. for i ← 0 to n - 2 do {
5.     min_index ← i
6.     for j ← i + 1 to n - 1 do {
7.         if (random_data[j] < random_data[min_index]) then {
8.             min_index ← j
9.         }
10.    }
11.    if (min_index ≠ i) then {
12.        temp ← random_data[i]
13.        random_data[i] ← random_data[min_index]
14.        random_data[min_index] ← temp
15.    }
16. }
17. return random_data
```

선택정렬 복잡도 분석

- 비교 횟수
 - ⊙ $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2 = O(n^2)$
- 이동 횟수
 - ⊙ $3(n - 1)$
- 전체 시간적 복잡도: $O(n^2)$
- 안정성을 만족하지 않음

삽입정렬(insertion sort)

- 정렬되어 있는 부분에 새로운 레코드를 올바른 위치에 삽입하는 과정 반복

a_0	a_1		a_{j-1}	b_j	b_{j+1}		b_{n-1}
오름차순 정렬 리스트				정렬되지 않은 리스트			

(그림 11.1) 삽입정렬의 현재 상태

a_0	a_1		a_j	b_{j+1}	b_{j+2}		b_{n-1}
오름차순 정렬 리스트				정렬되지 않은 리스트			

(그림 11.2) 삽입정렬의 다음 상태

삽입정렬(insertion sort)

○ 삽입정렬 과정

(표 11.1) 삽입정렬의 예

(1)	4	<u>2</u>	8	1	7	5
(2)	2	4	<u>8</u>	1	7	5
(3)	2	4	8	<u>1</u>	7	5
(4)	1	2	4	8	<u>7</u>	5
(5)	1	2	3	7	8	<u>5</u>
(6)	1	2	3	5	7	8

삽입정렬 알고리즘

1. Algorithm Insertion_sort(random_data, n)
2. Input: 양의 정수를 갖는 데이터 배열(random_data), 데이터 수(n)
3. Output: 오름차순으로 정렬된 배열(random_data)
4. for $i \leftarrow 1$ to $n-1$ do {
5. $key \leftarrow random_data[i]$
6. $index \leftarrow i-1$
7. while (($index \geq 0$) and ($random_data[index] > key$)) do {
8. $random_data[index+1] \leftarrow random_data[index]$
9. $index \leftarrow index-1$
10. }
11. $random_data[index+1] \leftarrow key$
12. }
13. return random_data

삽입정렬 복잡도 분석

- 최선의 경우 $O(n)$: 이미 정렬되어 있는 경우
 - ⊙ 비교: $n-1$ 번
- 최악의 경우 $O(n^2)$: 역순으로 정렬되어 있는 경우
 - ⊙ 모든 단계에서 앞에 놓인 자료 전부 이동
 - ⊙ 비교:
$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$$
 - ⊙ 이동:
$$\frac{n(n-1)}{2} + 2(n-1) = O(n^2)$$
- 평균의 경우 $O(n^2)$
- 많은 이동 필요 -> 레코드가 클 경우 불리
- 안정된 정렬방법
- 대부분 정렬되어 있으면 매우 효율적

버블정렬(bubble sort)

- 인접한 2개의 레코드를 비교하여 순서대로 되어 있지 않으면 서로 교환
- 이러한 비교-교환 과정을 리스트의 왼쪽 끝에서 오른쪽 끝까지 반복(스캔)



버블정렬 알고리즘

1. Algorithm Bubble_sort(random_data, n)
2. Input: 양의 정수를 갖는 데이터 배열(random_data), 데이터 수(n)
3. Output: 오름차순으로 정렬된 배열(random_data)
4. for $i \leftarrow 0$ to $n - 2$ do {
5. for $j \leftarrow 0$ to $n - 2 - i$ do {
6. if ($\text{random_data}[j] > \text{random_data}[j + 1]$) then {
7. $\text{temp} \leftarrow \text{random_data}[j]$
8. $\text{random_data}[j] \leftarrow \text{random_data}[j + 1]$
9. $\text{random_data}[j + 1] \leftarrow \text{temp}$
10. }
11. }
12. }
13. return random_data

버블정렬 복잡도 분석

- 비교 횟수(최상, 평균, 최악의 경우 모두 일정)

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$$

- 이동 횟수

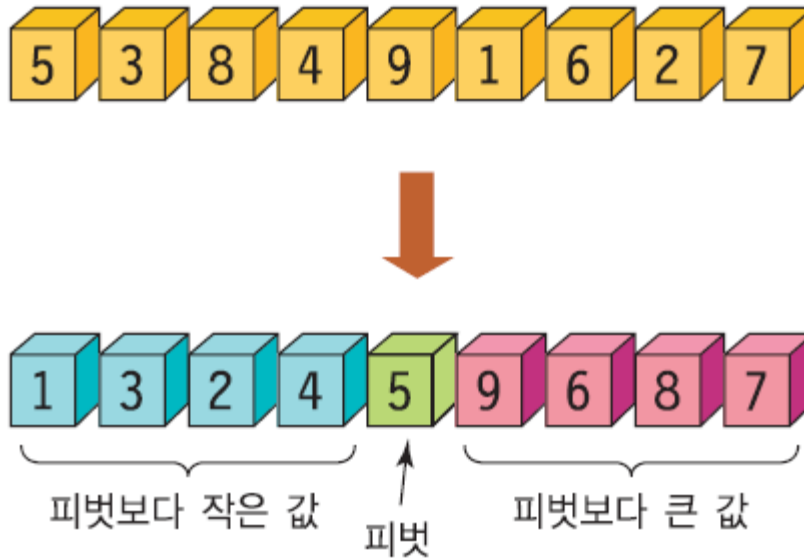
- ⊙ 역순으로 정렬된 경우(최악의 경우) : 이동 횟수 = 3 * 비교 횟수
- ⊙ 이미 정렬된 경우(최선의 경우) : 이동 횟수 = 0
- ⊙ 평균의 경우 : $O(n^2)$

- 레코드의 이동 과다

- ⊙ 이동연산은 비교연산 보다 더 많은 시간이 소요됨

퀵정렬(quick sort)

- 평균적으로 가장 빠른 정렬 방법
- 분할정복법 사용
- 리스트를 2개의 부분리스트로 비균등 분할하고, 각각의 부분리스트를 다시 퀵정렬함(재귀호출)



분할(partition)

- 피벗(pivot): 가장 왼쪽 숫자라고 가정
- 두개의 변수 low와 high를 사용한다.
- low는 피벗보다 작으면 통과, 크면 정지
- high는 피벗보다 크면 통과, 작으면 정지
- 정지된 위치의 숫자를 교환
- low와 high가 교차하면 종료

분할 과정

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	left	right
(1)	27	4	37	2	62	12	59	16	49	18	0	9
(2)	27	4	18	2	62	12	59	16	49	37		
(3)	27	4	18	2	16	12	59	62	49	37		
(4)	12	4	18	2	16	27	59	62	49	37	0	4
(5)	12	4	2	18	16	27	59	62	49	37		
(6)	2	4	12	18	16	27	59	62	49	37	0	1
(7)	2	4	12	18	16	27	59	62	49	37	3	4
(8)	2	4	12	16	18	27	59	62	49	37	6	9
(9)	2	4	12	16	18	27	59	37	49	62		
(10)	2	4	12	16	18	27	49	37	59	62	6	7
(11)	2	4	12	16	18	27	37	49	59	62	9	9
(12)	2	4	12	16	18	27	37	49	59	62		

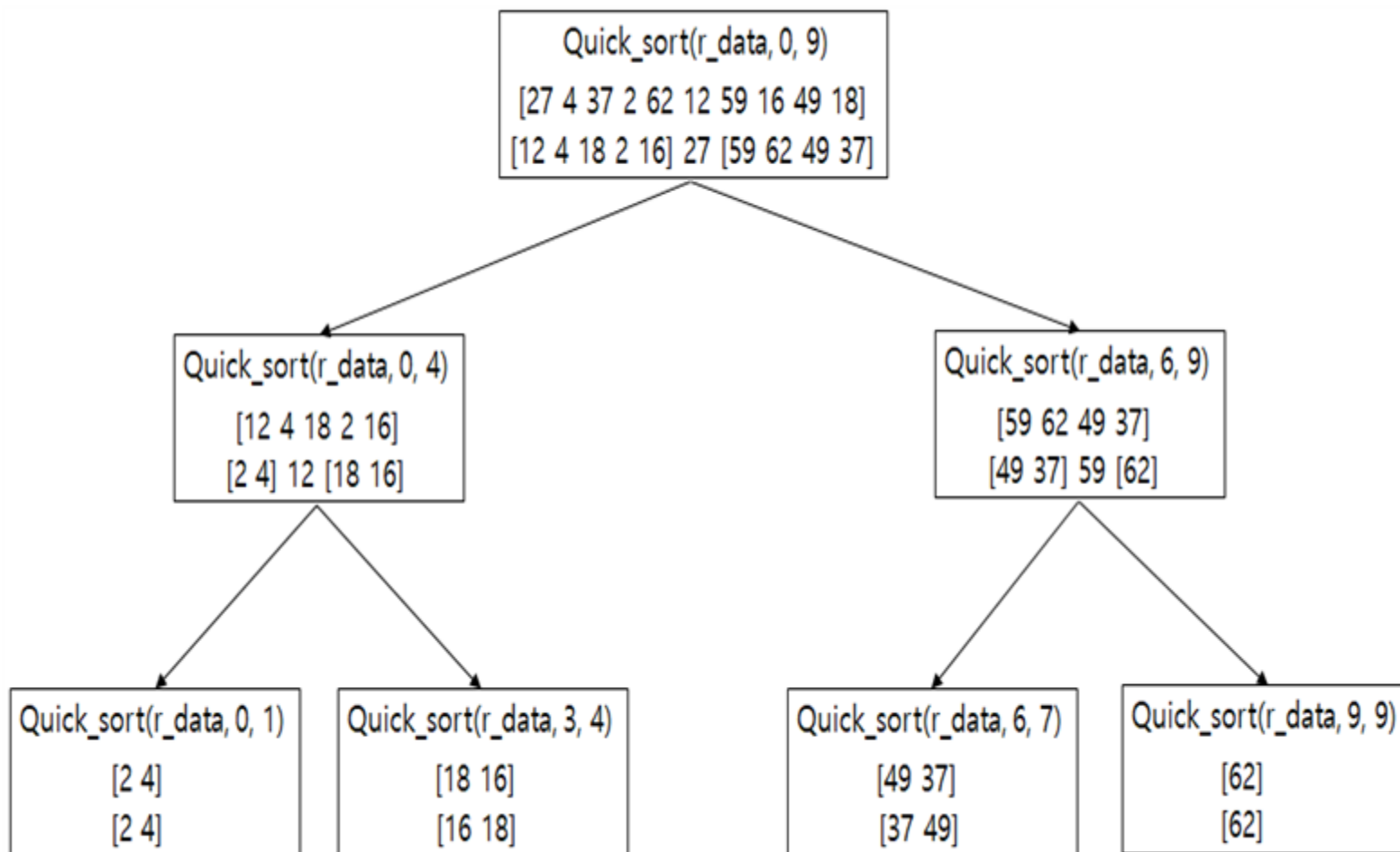
partition 함수

1. Algorithm Quick_partition(random_data, left, right)
2. Input: 랜덤 값을 갖는 데이터 배열(random_data), 배열에서 정렬을 위한 왼쪽(left)과 오른쪽(right) 인덱스
3. Output: 분할된 리스트에서 첫 번째 리스트의 마지막 인덱스(high)
4. low $\leftarrow$ left + 1
5. high $\leftarrow$ right
6. pivot $\leftarrow$ random_data[left]
7. while (low $\leq$ hi) do {
 8. while (random_data[high] $>$ pivot) do
 9. high $\leftarrow$ high - 1
 10. while (low $\leq$ high and random_data[low] $\leq$ pivot) do
 11. low $\leftarrow$ low + 1
 12. if (low $\leq$ high) {
 13. swap(random_data[low], random_data[high])
 14. low $\leftarrow$ low + 1
 15. high $\leftarrow$ high - 1 }
 16. }
 17. swap(random_data[left], random_data[high])
 18. return high

Quick_sort 함수

1. Algorithm Quick_sort(random_data, left, right)
2. Input: 랜덤 값을 갖는 데이터 배열(random_data), 배열에서 정렬을 위한 왼쪽(left)과 오른쪽(right) 인덱스
3. Output: 오름차순으로 정렬된 리스트(random_data)
4. if (left < right) then {
5. q <- Quick_partition(random_data, left, right)
6. Quick_sort(random_data, left, q-1)
7. Quick_sort(random_data, q+1, right) }
8. end

퀵정렬 전체 과정



퀵 정렬 복잡도 분석

- 최선의 경우(거의 균등한 리스트로 분할되는 경우)

- 패스 수: $\log(n)$

- 2→1

- 4→2

- 8→3

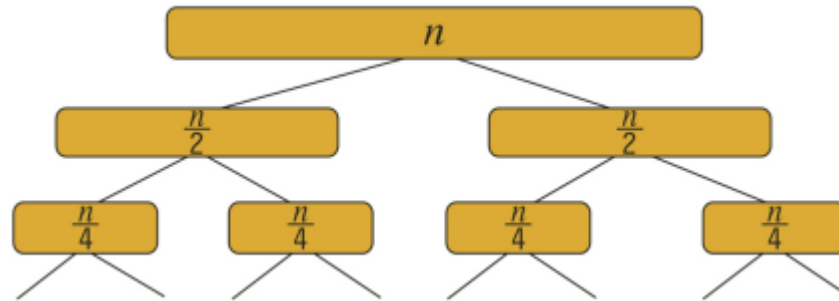
- ...

- $n \rightarrow \log(n)$

- 각 패스 안에서의 비교횟수: n

- 총 비교횟수: $n * \log(n)$

- 총 이동횟수: 비교횟수에 비하여 적으므로 무시 가능

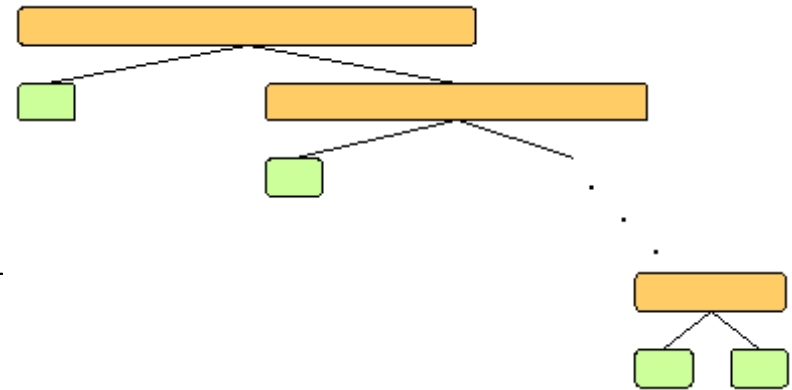


퀵정렬 복잡도 분석(cont.)

- 최악의 경우(극도로 불균등한 리스트로 분할되는 경우)

- ⦿ 패스 수: n
- ⦿ 각 패스 안에서의 비교횟수: n
- ⦿ 총 비교횟수: n^2
- ⦿ 총 이동횟수: 무시 가능
- ⦿ (예) 이미 정렬된 리스트를 정렬할

(1	2	3	4	5	6	7	8	9)
1	(2	3	4	5	6	7	8	9)
1	2	(3	4	5	6	7	8	9)
1	2	3	(4	5	6	7	8	9)
1	2	3	4	(5	6	7	8	9)
				...				
1	2	3	4	5	6	7	8	9



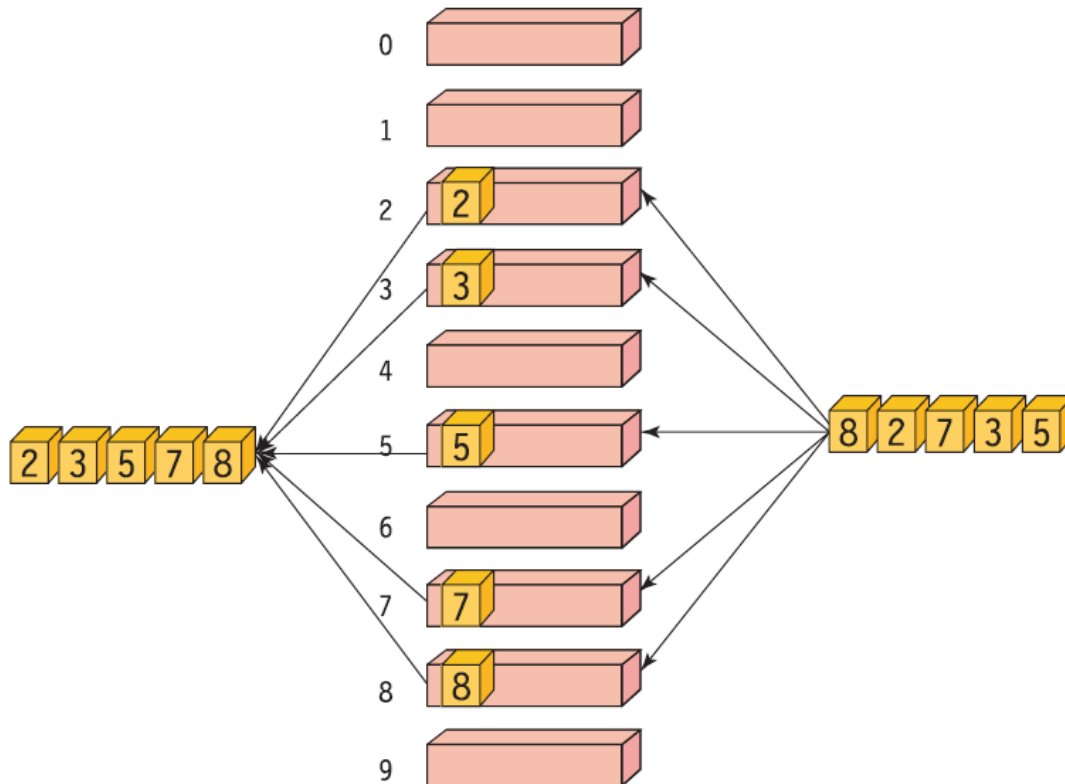
- ⦿ 중간값(**medium**)을 피벗으로 선택하면 불균등 분할 완화 가능

기수정렬(Radix Sort)

- 대부분의 정렬 방법들은 레코드들을 비교함으로써 정렬 수행
- 기수 정렬(**radix sort**)은 레코드를 비교하지 않고 정렬 수행
 - ⊙ 비교에 의한 정렬의 하한인 $O(n \cdot \log(n))$ 보다 좋을 수 있음
 - ⊙ 기수 정렬은 $O(dn)$ 의 시간적복잡도를 가짐(대부분 $d < 10$ 이하)
- 기수 정렬의 단점
 - ⊙ 정렬할 수 있는 레코드의 타입 한정(실수, 한글, 한자 등은 정렬 불가)
 - ⊙ 즉, 레코드의 키들이 동일한 길이를 가지는 숫자나 단순 문자(알파벳 등)이어야만 함

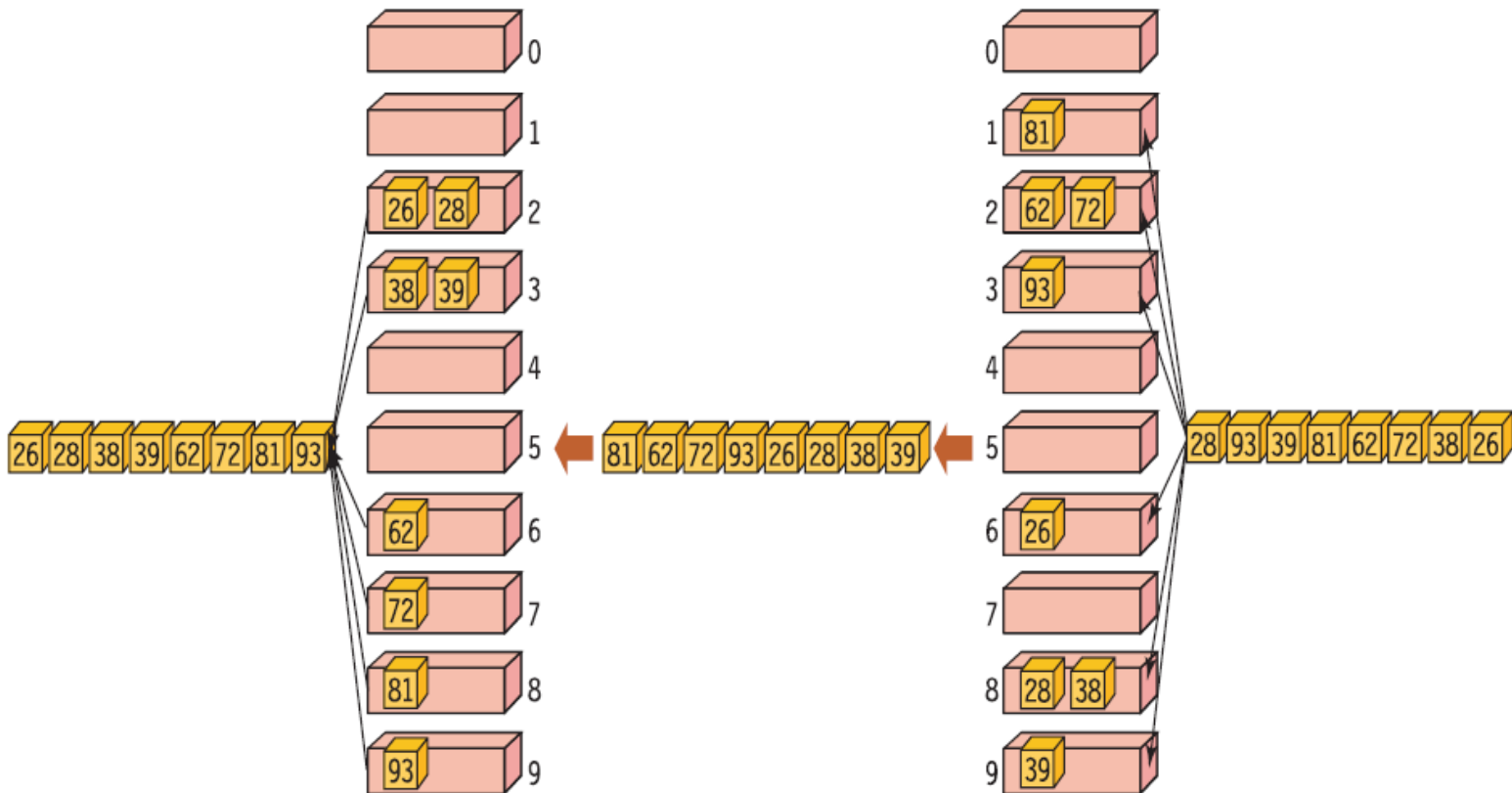
기수정렬

- (예) 한자리수 (8, 2, 7, 3, 5)의 기수정렬
- 단순히 자리수에 따라 버킷(bucket)에 넣었다가 꺼내면 정렬됨



기수정렬

- 만약 2자리수이면? (28, 93, 39, 81, 62, 72, 38, 26)
- 낮은 자리수로 먼저 분류한 다음, 순서대로 읽어서 다시 높은 자리수로 분류



기수정렬 알고리즘

- 기수정렬은 큐로 구현
- 기수정렬의 개수는 키의 표현 방법과 밀접한 관계
 - ⊙ 이진법을 사용한다면 버킷은 2개.
 - ⊙ 알파벳 문자를 사용한다면 버킷은 26개
 - ⊙ 십진법을 사용한다면 버킷은 10개
- (예) 32비트의 정수의 경우, 8비트씩 나누면 → 버킷은 256개로 늘어남. 대신 필요한 패스의 수는 4로 줄어듦.

기수정렬 알고리즘

```
1. Algorithm Radix_sort(random_data, n)
2. Input: 양의 정수를 갖는 데이터 배열(random_data), 데이터 수(n)
3. Output: 오름차순으로 정렬된 배열(random_data)
4. max_value ← 최대값(random_data)
5. digit ← 1
6.
7. while (max_value / digit > 0) do {
8.     create 10개의 큐 bucket[0..9]
9.
10.    for i ← 0 to n - 1 do {
11.        index ← (random_data[i] / digit) mod 10
12.        bucket[index].enqueue(random_data[i])
13.    }
14.
15.    k ← 0
16.    for b ← 0 to 9 do {
17.        while (bucket[b] is not empty) do {
18.            random_data[k] ← bucket[b].dequeue()
19.            k ← k + 1
20.        }
21.    }
22.
23.    digit ← digit × 10
24. }
25. return random_data
```

기수정렬 복잡도 분석

- n 개의 레코드, d 개의 자릿수로 이루어진 키를 기수 정렬할 경우
 - ⊙ 메인 루프는 자릿수 d 번 반복
 - ⊙ 큐에 n 개 레코드 입력 수행
- $O(dn)$ 의 시간적 복잡도
 - ⊙ 키의 자릿수 d 는 10 이하의 작은 수이므로 빠른 정렬임
- 실수, 한글, 한자로 이루어진 키는 정렬 못함

정렬 알고리즘의 비교

알고리즘	최선	평균	최악
삽입 정렬	$O(n)$	$O(n^2)$	$O(n^2)$
선택 정렬	$O(n^2)$	$O(n^2)$	$O(n^2)$
버블 정렬	$O(n^2)$	$O(n^2)$	$O(n^2)$
셸 정렬	$O(n)$	$O(n^{1.5})$	$O(n^{1.5})$
퀵 정렬	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$
힙 정렬	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$
합병 정렬	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$
기수 정렬	$O(dn)$	$O(dn)$	$O(dn)$

정렬 알고리즘의 실험 예 (정수 60,000개)

알고리즘	실행 시간(단위:sec)
삽입 정렬	7.438
선택 정렬	10.842
버블 정렬	22.894
셸 정렬	0.056
히프 정렬	0.034
합병 정렬	0.026
퀵 정렬	0.014